

DL for ENG

## LECTURE 4.1

# From Perceptron to Neural Network

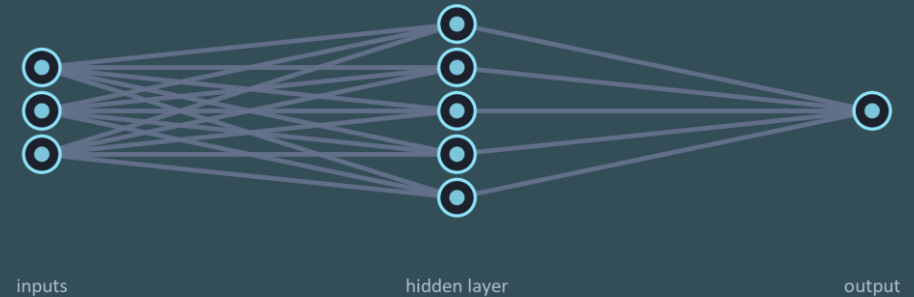
**Remus Teodorescu**

With support from Research Assistant Noman Khan -  
[nomank@energy.aau.dk](mailto:nomank@energy.aau.dk)

Aalborg University · [ret@et.aau.dk](mailto:ret@et.aau.dk)

Deep Learning for Engineering · 2026

### NEURON · HIDDEN LAYER · APPROXIMATION



*A neuron is a weighted sum, a bias and an activation, and one neuron draws one straight boundary: enough for AND and OR, never for XOR. Put a hidden layer of neurons between input and output and the boundary can bend; with enough neurons the network approximates any continuous function, one linear region at a time.*

# Learning Objectives

## SECTIONS, IN ORDER

- 1 **Supervised learning** — data, model and loss; linear regression and its loss surface
- 2 **Neuron** — the perceptron and the neuron, linear separability, the hidden layer, the activation
- 3 **Neural network** — one hidden layer, linear regions, universal approximation, sizing

*Every curve in this lecture is already fitted: the loss and the optimiser are held fixed, and only the function is studied.*

## READ, AND WHAT EACH ANSWERS

**UDL ch. 2**      how is a model fitted to data?

**UDL ch. 3.1**      what does one neuron compute, and what can it not?

**UDL ch. 3.2 – 3.5**      what can one hidden layer represent?

# Recap

## 1 – 5 · WHAT IT HAS BUILT

- 1 What are the five questions to ask of any reported result, and what does each one test? *UDL ch. 1.4*
- 2 What are the three vision tasks, and why does the third need far more labelling than the first? *UDL ch. 10.5*
- 3 What does a language model compute at each step, and what is the exportable idea behind it? *UDL ch. 12.5 – 12.8*
- 4 What problem did AlphaFold solve, what data did it learn from, and what does it still not give you? *Jumper et al. (2021), Nature 596*
- 5 What is GraphCast, what baseline did it beat, and what does it depend on at every single run? *Lam et al. (2023), Science 382*

## 6 – 10 · WHAT AN ENGINEER BUILDS

- 6 What did the plasma-control result show, and which of the five questions does it answer least well? *Degrave et al. (2022), Nature 602*
- 7 What is a surrogate model, when is training one worth the cost, and where does it stop being trustworthy? *UDL ch. 1.4*
- 8 Where does the data for predictive maintenance come from, and why is the base rate the real difficulty? *UDL ch. 1.4*
- 9 What does a network contribute to design search, and what is still doing the engineering? *UDL ch. 14 – 18*
- 10 What does it cost, which of those costs apply to the models in this course, and which do not? *UDL ch. 1.4*

# Supervised Learning

**Supervised learning** — building a model  $f$  that maps an input  $x$  to a prediction  $y$ , from a training set of measured input–output pairs. Training chooses the model's parameters  $\theta$  to make the loss, one number that measures the mismatch, as small as possible; the fitted model is then checked on test data it has not seen.

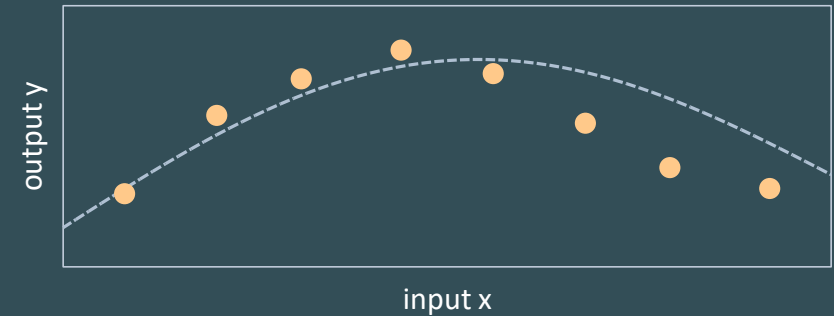
## THREE OBJECTS

- **the data** —  $N$  measured input–output pairs
- **the model** — input and parameters in, prediction out
- **the loss** — one number for the whole mismatch

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_{k=1}^N \ell(f(x_k, \theta), y_k)$$

$\hat{\theta}$  the fitted parameters,  $f$  the model,  $\ell$  the loss,  $(x_k, y_k)$  the  $k$ th of  $N$  pairs

## DATA AND THE FUNCTION BEHIND IT



*Amber: the measured pairs; dashed: the function.*

## SUPERVISED LEARNING IN PYTORCH

```
model = nn.Linear(1, 1)      # f, with parameters  $\theta$ 
y_hat = model(x)             # a prediction per input
loss = ((y_hat - y)**2).mean() # one number: the mismatch
loss.backward(); opt.step()  # move  $\theta$  to lower it
```

# Linear Regression

**Linear regression** — the simplest model: the prediction is a straight line,  $y = \theta_0 + \theta_1 x$ , with the intercept and the slope as its two parameters. Least squares chooses them by making the sum of the squared differences between prediction and measurement as small as possible.

## MODEL AND LOSS

- **two parameters** — the intercept and the slope
- **least squares** — the summed squared residuals
- **the baseline** — for every model after it

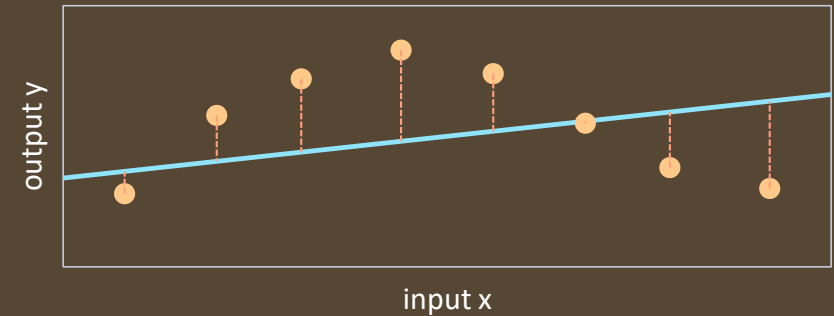
$$f(x, \theta) = \theta_0 + \theta_1 x$$

$x$  the input,  $f$  the prediction,  $\theta_0$  the intercept,  $\theta_1$  the slope

$$\mathcal{L}(\theta) = \sum_{k=1}^N (\theta_0 + \theta_1 x_k - y_k)^2$$

$\mathcal{L}$  the loss,  $\theta_0 + \theta_1 x_k$  from the model,  $y_k$  from the label,  $N$  data points

## THE FIT AND ITS RESIDUALS



Orange dashes: the residuals, as small as a line allows.

## LINEAR REGRESSION IN PYTORCH

```
line = nn.Linear(1, 1)           #  $\theta_0$  the bias,  $\theta_1$  the weight
loss = ((line(x) - y)**2).sum()  # least squares
theta = torch.linalg.lstsq(X, y).solution # or exactly;  $X = [1, x]$ 
```

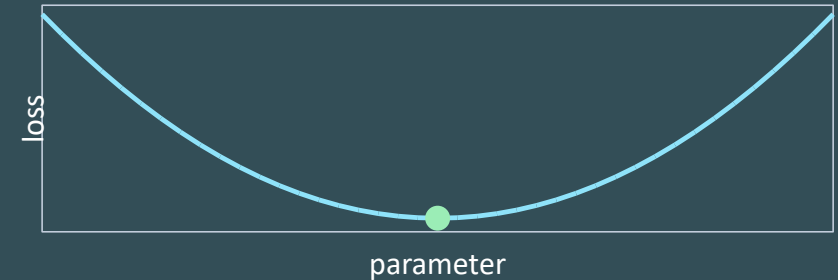
# Loss Surface

**Loss surface** — the loss drawn as a function of the parameters: every choice of parameters is a point, and its height is the loss. Training is a search for the lowest point. For linear regression the surface is a bowl with one minimum and a formula for it; for a neural network it has many minima and must be searched step by step.

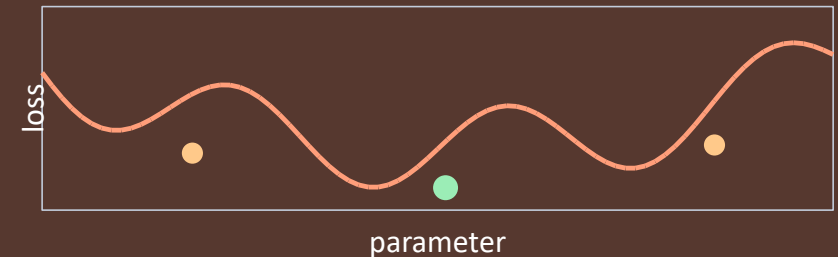
## THE BOWL AND THE RUGGED SURFACE

- **linear regression** — a bowl, one minimum, a formula
- **a neural network** — many minima, no formula
- **so the minimum is searched for** — step by step
- **the picture to keep** — the bowl is the exception

### LINEAR MODEL — ONE MINIMUM



### NEURAL NETWORK — MANY MINIMA



# Perceptron and Neuron

**Perceptron and neuron** — a neuron takes a weighted sum of its inputs plus a bias and passes it through an activation function. The perceptron of 1958 is the neuron with a step activation: it outputs a class and learns by correcting its errors. The modern neuron's activation is continuous, so gradients can train it.

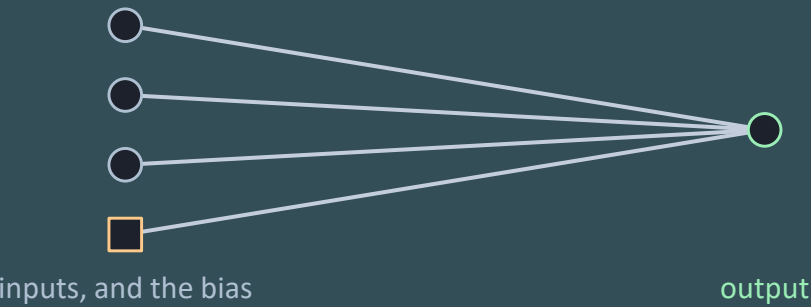
## THE 1958 UNIT AND THE MODERN ONE

|            | PERCEPTRON       | NEURON                 |
|------------|------------------|------------------------|
| activation | step: 0 or 1     | continuous: ReLU, tanh |
| output     | a class          | a real number          |
| learning   | error correction | gradient descent       |

$$y = H(\mathbf{w} \cdot \mathbf{x} + b)$$

$\mathbf{x}$  the inputs,  $\mathbf{w}$  the weights,  $b$  the bias,  $H$  the step function,  $y \in \{0, 1\}$

### ONE NEURON



### PERCEPTRON AND NEURON IN PYTORCH

```
z = nn.Linear(3, 1)(x)      # w · x + b: three weights, a bias
y = (z > 0).float()          # perceptron: the step H
a = torch.tanh(z)            # neuron: a smooth activation
```

# Linear Separability

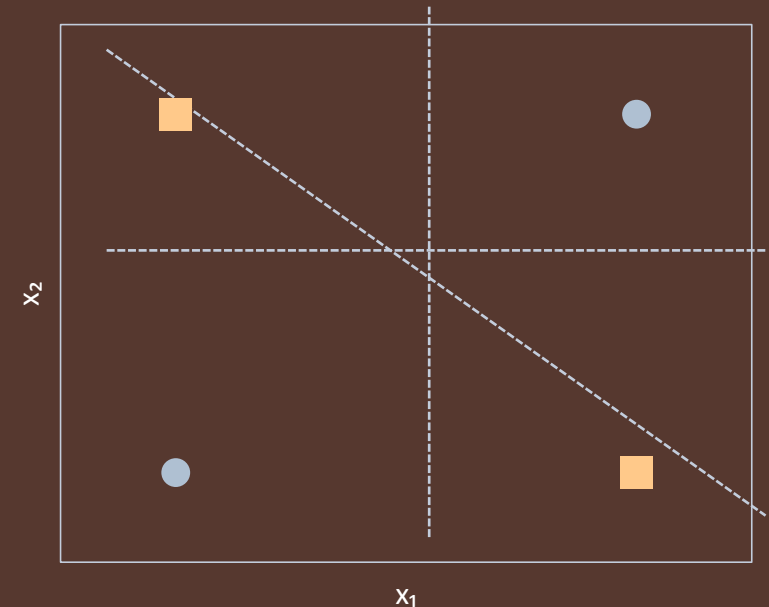
**Linear separability** — two classes are linearly separable when one straight line (a plane, with more inputs) puts every sample of one class on one side and every sample of the other on the other side. One neuron draws exactly one such boundary, so it can learn AND and OR, but not XOR.

## ONE NEURON, ONE STRAIGHT BOUNDARY

- **the weighted sum is zero along a line** — a class each side
- **AND and OR** — separable: one neuron suffices
- **XOR** — opposite diagonals: no line separates them

**TALK · 2 MIN** How would you explain linear separability to a colleague, in your own words? Then, in the XOR plot on the right, why does each dashed line leave an amber square and a grey circle on the same side?

## XOR — NOT SEPARABLE



*Amber squares are class 1, grey circles class 0. Every straight line misclassifies at least one point.*

# Hidden Layer

**Hidden layer** — a layer of neurons between the inputs and the output. Each hidden neuron draws its own boundary, and the output neuron combines their decisions, so the network is no longer limited to one straight line: two hidden neurons are enough for XOR.

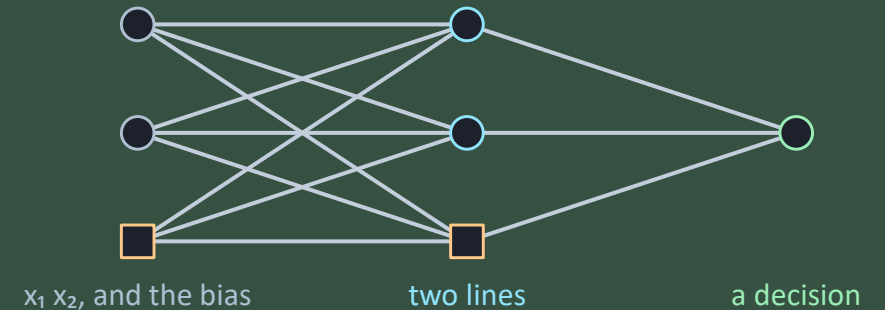
## TWO LINES, THEN A DECISION ABOUT THEM

- **two hidden neurons** — two lines
- **the output neuron** — a line in the hidden decisions
- **the feature  $x_1x_2$**  — does the same, by hand

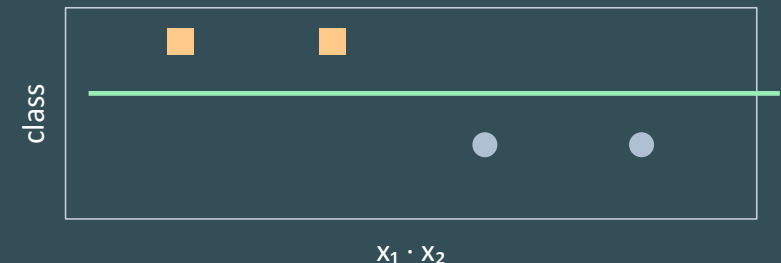
### THE XOR NETWORK IN PYTORCH

```
net = nn.Sequential(nn.Linear(2, 2), # two hidden neurons, two lines
                    nn.ReLU(),       # the activation between
                    nn.Linear(2, 1)) # the decision: nine parameters in all
```

## TWO INPUTS, TWO HIDDEN, ONE OUTPUT



## XOR WITH THE FEATURE $x_1x_2$

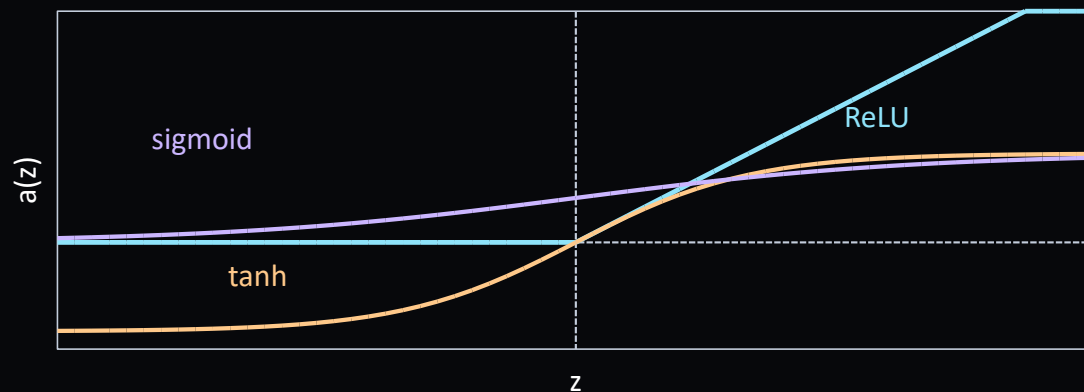


# Activation Function

**Activation function** — the nonlinear function applied to each neuron's weighted sum. Without it, layers of weighted sums collapse into one linear function, however many there are; with it, the network can bend. ReLU keeps positive values and sets the rest to zero; tanh and sigmoid are smooth S-shapes.

## WHY A NONLINEARITY, AND WHICH ONE

- **without it** — layers collapse to one linear function
- **the choice** — set by the derivatives the loss needs



## FUNCTIONS AND DERIVATIVES

|         | $a(z)$             | $a'(z)$               | $a''(z)$          |
|---------|--------------------|-----------------------|-------------------|
| ReLU    | $\max(0, z)$       | 0 or 1                | 0                 |
| tanh    | $\tanh z$          | $1 - \tanh^2 z$       | $\neq 0$ , smooth |
| sigmoid | $1 / (1 + e^{-z})$ | $\sigma (1 - \sigma)$ | $\neq 0$ , smooth |

## WHERE EACH IS USED

|                |                                   |
|----------------|-----------------------------------|
| <b>ReLU</b>    | classification and regression     |
| <b>tanh</b>    | PDE residuals: second derivatives |
| <b>sigmoid</b> | a probability at the output       |

*A PDE residual holds second derivatives: zero along a ReLU piece, nonzero for tanh.*

# Shallow Neural Network

**Shallow neural network** — a network with one hidden layer. Each of its  $D$  hidden neurons computes a weighted sum of the input plus a bias and applies the activation; the output is a weighted sum of the  $D$  results plus a bias. It is still a function  $y = f(x, \theta)$ , with many more parameters than a line.

## ONE HIDDEN LAYER, $D$ NEURONS

- **each hidden neuron** — scale, shift, activate
- **the output** — a weighted sum of the  $D$  results

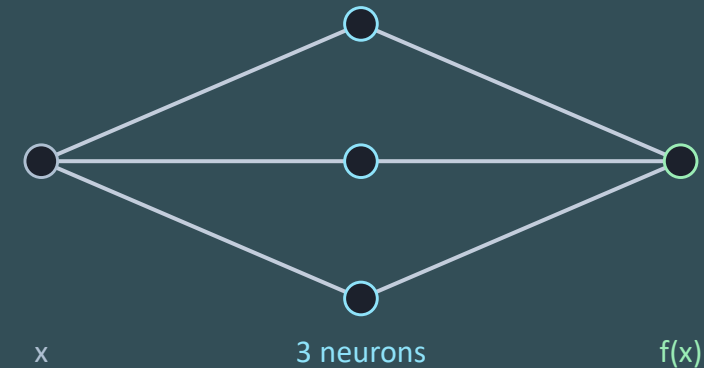
$$f(x, \theta) = \varphi_0 + \sum_{d=1}^D \varphi_d a(\theta_{d0} + \theta_{d1}x)$$

$D$  hidden neurons,  $\theta_{d0}, \theta_{d1}$  the bias and weight of neuron  $d$ ,  $\varphi_0, \varphi_d$  the output bias and weights

### A SHALLOW NETWORK IN PYTORCH

```
net = nn.Sequential(nn.Linear(1, D), # θ: D weights, D biases
                    nn.ReLU(),       # a(·), on each neuron
                    nn.Linear(D, 1)) # φ: D weights, one bias
sum(p.numel() for p in net.parameters()) # P = 3D + 1
```

## ONE INPUT, THREE HIDDEN, ONE OUTPUT



## PARAMETER COUNT

$$P = D(D_i + 1) + D_o(D + 1)$$

**D = 10**

31 parameters, one in, one out

# Linear Regions

**Linear regions** — the pieces of the input on which a ReLU network is exactly linear. Each hidden neuron switches on at one point, a breakpoint, so with one input and  $D$  hidden neurons the network has at most  $D$  breakpoints and  $D + 1$  linear regions. Training moves the breakpoints.

## $D$ NEURONS, $D + 1$ LINEAR REGIONS

- **each ReLU neuron** — one breakpoint
- **training** — moves the breakpoints
- **$D = 10$**  — still straight pieces, up close

**TALK · 2 MIN** How would you explain what a ReLU network computes, in your own words? Then, comparing the two fits on the right, how many linear regions has the  $D = 10$  network, and where does it still miss the dashed target?

*Ex\_04 notebook 03 sweeps  $D$  and records the error at each width.*

## $D = 3$ — FOUR REGIONS



## $D = 10$ — ELEVEN REGIONS



# Universal Approximation

**Universal approximation** — the theorem that a network with one hidden layer and enough neurons can approximate any continuous function on a closed, bounded region to any accuracy you ask for. It says such a network exists; it does not say how many neurons it needs or how to find its parameters.

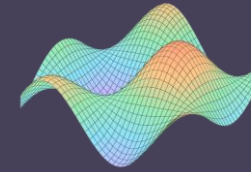
## PIECEWISE APPROXIMATION

- **enough linear regions** — any continuous function
- **the analogy** — finer steps, smaller error, more of them
- **what it does not give** — the width, or the parameters

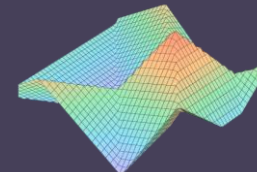
### WHAT IT DOES NOT SAY

No theory gives the width or the depth. Ex\_04 notebook 04 sweeps both at equal parameter count.

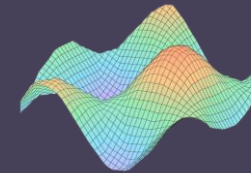
## ONE LAYER OR TWO



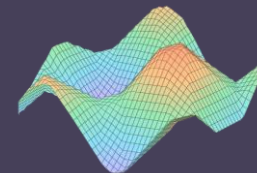
the target function  
 $f(x, y)$  on the unit square



**8 neurons, 1 layer**  
33 parameters · RMSE 0.102



**32 neurons, 1 layer**  
129 parameters · RMSE 0.017



**8 + 8 neurons, 2 layers**  
105 parameters · RMSE 0.023

# Inputs and Outputs

**Inputs and outputs** — a network can take several inputs and give several outputs. With several inputs each hidden neuron divides the input space along a plane instead of at a point; several outputs are several weighted sums of the same hidden neurons, so they share what the hidden layer computes.

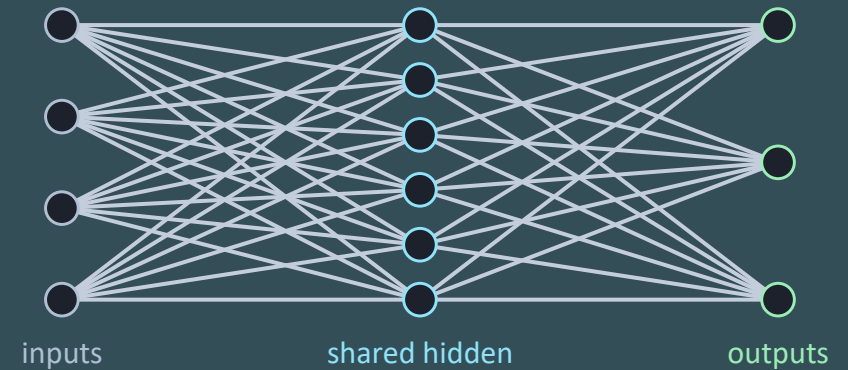
## SEVERAL OF EACH

- **the inputs** — what the quantity depends on
- **the outputs** — what is needed together
- **the parameters** — grow with all three sizes

### FOUR IN, THREE OUT, IN PYTORCH

```
net = nn.Sequential(nn.Linear(4, 6), # four inputs, six hidden neurons
                    nn.ReLU(),       # the activation
                    nn.Linear(6, 3)) # three outputs, one shared layer
```

## FOUR IN, THREE OUT



## SHARED HIDDEN LAYER

Outputs that describe one physical state come from one network. The hidden layer is shared, so the outputs are coupled the way the physics couples them.

# Sizing a Network

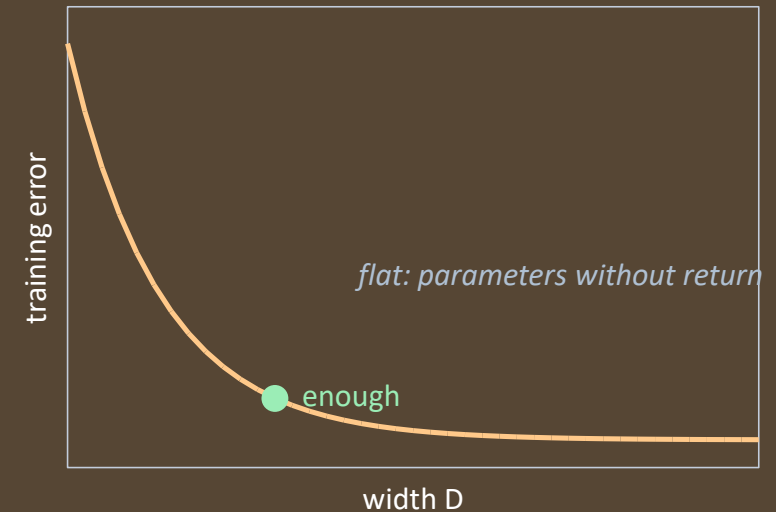
**Sizing a network** — choosing its width, the number of neurons in a layer, and its depth, the number of hidden layers. No theory gives either. Start with the smallest network that can fit the data, and make it larger only while the error still falls.

## WIDTH D AND DEPTH N

- **the parameters** — grow as  $N \cdot D^2$
- **start small** — tens of neurons, not thousands
- **grow** — only while the training error falls
- **past the elbow** — parameters without return

*Ex\_04 notebook 04 compares deeper and wider at equal parameter count.*

## ERROR AGAINST WIDTH, SCHEMATIC



*Schematic, no units. The curve flattens; the elbow is where a first network should sit, and the notebook finds it for a concrete problem.*

# Advanced Topics

## RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

| topic                | what it adds                     | read            | where it makes a difference |
|----------------------|----------------------------------|-----------------|-----------------------------|
| generative models    | predict the reading, then invert | UDL ch. 2 notes | known physics, pose         |
| leaky ReLU, GELU     | no zero slope below zero         | UDL ch. 3 notes | large vision models         |
| dying ReLU           | a neuron that never switches on  | UDL ch. 3 notes | large ReLU networks         |
| regions, many inputs | how D planes cut the space       | UDL ch. 3 notes | theory of capacity          |
| the perceptron rule  | learning without gradients       | UDL ch. 3 notes | history, one neuron         |

### WHY THEY ARE LEFT OUT

Each refines today's neuron or network, and most pay off only in very large models. ReLU, tanh and one or two hidden layers are what this course's engineering problems need.

# Exercise 4

## THE NOTEBOOKS, IN ORDER

- **00** — checks the setup
- **01** — a perceptron in NumPy: AND, then XOR, then a hidden layer set by hand
- **02** — the same network in PyTorch, trained by gradient descent
- **03** — sweep the width  $D$ , count the linear regions, record the error
- **04 to 06** — after L4.2: depth against width, overfitting, and the report

## THE QUESTION EACH NOTEBOOK ANSWERS

- 01**      why can one neuron not learn XOR?
- 02**      what did autograd replace?
- 03**      how does the error fall with  $D$ ?

## REPORT QUESTION, TO CONCLUDE

Across the notebooks: what decided how well your network fitted — the activation, the width, the depth or the data — and what number shows it?

# Key Takeaways

## NEURON

|                    |                                                |
|--------------------|------------------------------------------------|
| <b>what it is</b>  | a weighted sum, a bias and an activation       |
| <b>function</b>    | one boundary: a line, or a plane               |
| <b>train, tune</b> | a continuous activation, so gradients work     |
| <b>vs linear</b>   | linear regression is one neuron, no activation |
| <b>used for</b>    | AND and OR, never XOR; every network's unit    |

## NEURAL NETWORK

|                      |                                                |
|----------------------|------------------------------------------------|
| <b>what it is</b>    | a hidden layer of neurons, then a weighted sum |
| <b>function</b>      | D ReLU neurons: $D + 1$ linear regions         |
| <b>train, tune</b>   | start small; grow while the error falls        |
| <b>vs one neuron</b> | XOR with two hidden neurons, nine parameters   |
| <b>used for</b>      | any continuous function, with enough neurons   |

*Next — L4.2: deep networks, capacity and generalisation.*

# Questions

## 1 – 5 · WHAT IT HAS BUILT

- 1 What is a neuron? Write its equation and name each term in it. *UDL ch. 3.1*
- 2 What function does one neuron implement, and why can it not represent XOR? *GBC ch. 6.1; Ex\_04 notebook 01*
- 3 How is a neuron trained: what separates it from a perceptron, and why does that matter for training? *GBC ch. 6.1; Ex\_04 notebook 02*
- 4 How does a neuron differ from linear regression, and why does a network without activations stay linear? *UDL ch. 3.1*
- 5 What is it used for: which activation for a regression, and which when second derivatives enter the loss? *UDL ch. 3.1; Ex\_04 notebook 03*

## 6 – 10 · WHAT AN ENGINEER BUILDS

- 6 What is a shallow neural network, and how many parameters has one with  $D_i$  inputs,  $D$  neurons and  $D_o$  outputs? *UDL ch. 3.3 – 3.4*
- 7 What function does a ReLU network with  $D$  hidden neurons compute, and how many linear regions has it? *UDL ch. 3.2; Ex\_04 notebook 03*
- 8 How is it sized: the width and depth of a first network, and what result would make you change them? *UDL ch. 4.5; Ex\_04 notebook 04*
- 9 How does it differ from one neuron: what do two hidden neurons change for XOR, and why share one hidden layer? *GBC ch. 6.1; UDL ch. 3.4*
- 10 What does the universal approximation theorem state, and what are two things it does not tell you? *UDL ch. 3.2; Ex\_04 notebook 03*

NEXT

# L4.2 — Deep Neural Networks

---

## BEFORE THE NEXT SESSION

|                    |                                  |
|--------------------|----------------------------------|
| <b>read</b>        | UDL ch. 4 — about forty minutes  |
| <b>run</b>         | Ex_04 notebooks 00 to 03         |
| <b>bring</b>       | the epoch count where XOR failed |
| <b>think about</b> | why nine parameters were enough  |

*Bring the number. L4.2 opens with a show of hands on how long people let the single-layer version run before concluding it would never converge.*